

ELTE EÖTVÖS LORÁND UNIVERSITY

USING & TESTING WEB SERVICES

Modern web application development in .NET

Lecture 5

Dr. Máté Cserép

mcserep@inf.elte.hu

https://mcserep.web.elte.hu/

WEB SERVICES

Web services enable platform-independent data exchange between applications built on possibly different technologies

- the most common model is *REST (Representational State Transfer)*, which uses the HTTP protocol for communication
- the service can be implemented as an ASP.NET Core MVC application, the client can be implemented in an arbitrary language
- operations can communicate through not only primitive types, but also complex *Data Transfer Objects (DTOs)*
- *JSON (Javascript Object Notation)* is the most common format for object-oriented data transfer, as it is easy to interpret both by machines and humans

.NET CLIENTS

ASP.NET Core web services can be accessed as a consumer using the classes in the **`System.Net.Http`** namespace

The **`HttpClient`** type provides the connection to HTTP-based services

- since network communication can be time consuming, it provides asynchronous functions to execute HTTP commands (**`GetAsync`**, **`PutAsync`**, etc.)
- the result of these method calls contains the response (**`HttpResponseMessage`**)
- when the operation is successful (**`IsSuccessStatusCode`**), the content of the response can be processed (**`Content`** property)

.NET CLIENTS

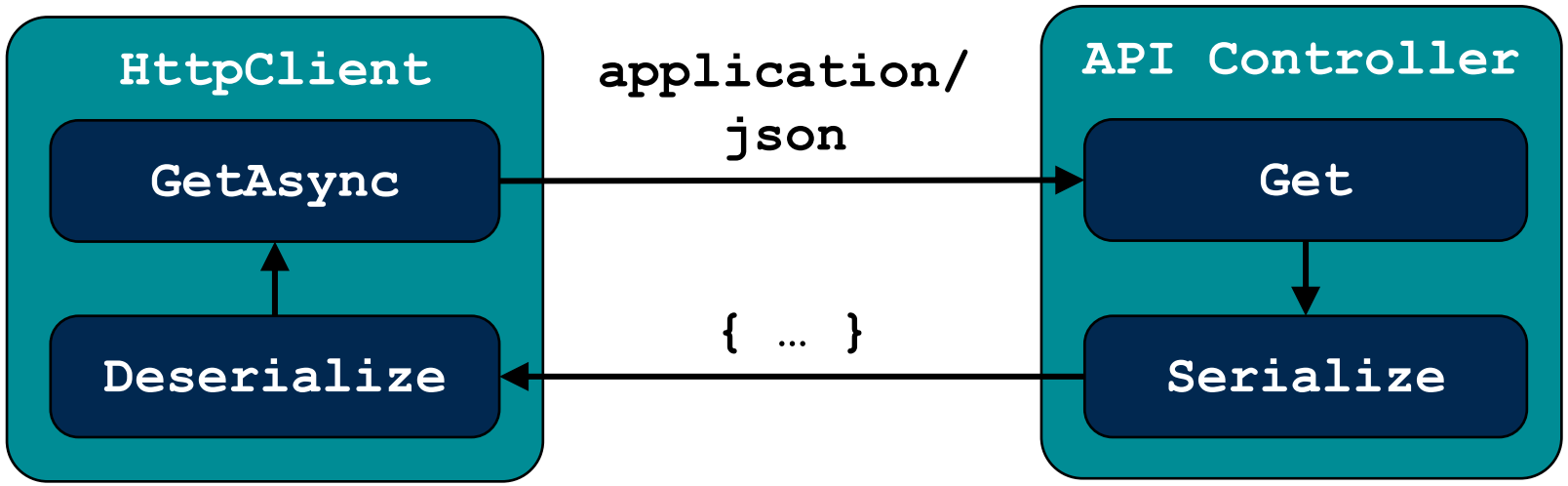
- e.g.:

```
using (HttpClient client = new HttpClient())
// instantiate new client
{
  HttpResponseMessage response =
    await client.GetAsync("http://…");
    // execute asynchronous query
  if (response.IsSuccessStatusCode)
  {
    HttpContent content = response.Content;
    ... // process content
  }
} // destroy client
```

DATA MANAGEMENT IN .NET CLIENTS

Content is managed in an object-oriented way, the format of the data transfer is controlled by the framework

- the way the data transfer object is *serialized* and *deserialized* is done according to the HTTP header information



DATA MANAGEMENT IN .NET CLIENTS

- this requires the data transfer objects to have the same type on both the client and server side
  - best practice is to reference them from the same class library
- reading the response, only the data type needs to be specified, e.g.:

```
if (response.IsSuccessStatusCode) {
  Product myProduct = await
    response.Content.ReadAsAsync<ProductDto>();
  ... // process content
}
```

- we also have the option to handle the received content as text (**ReadAsStringAsync**), binary (**ReadAsByteArrayAsync**) or stream (**ReadAsStreamAsync**) data

DATA MANAGEMENT IN .NET CLIENTS

- serialization can be customized through the specialization of the abstract **`MediaTypeFormatter`** base type, so that specific message formats can be created
- when content is also sent (e.g. **`POST`**, **`PUT`**),
  - its format can be specified with the proper formatter, e.g.:
    ```
    client.PostAsync("http://...", product,
      new JsonMediaTypeFormatter());
      // serialize the data to JSON
    ```
  - alternatively, extension methods (provided by the *Microsoft.AspNet.WebApi.Client* NuGet package), can be used, e.g.:
    ```
    client.PostAsJsonAsync("http://...", product);
    // serialize the data to JSON
    ```

DATA MANAGEMENT IN .NET CLIENTS

Key configuration available on the **`HttpClient`** object:

- the prefix for all API calls (**`BaseAddress`**),
- the timeout for communication (**`Timeout`**),
- the default headers for each message sent (**`DefaultRequestHeaders`**), and within that
  - the content format (**`DefaultRequestHeaders.Accept`**), e.g:

```
client.DefaultRequestHeaders.Accept.Clear();
client.DefaultRequestHeaders.Accept.Add(new
  MediaTypeWithQualityHeaderValue("application/json"));
  // the client will only accept JSON format
```

CRUD

In many cases the web service provides basic data processing operations, the *CRUD* operations: *Create, Read, Update, Delete*

- these operations have HTTP equivalents (create: **`POST`**, read: **`GET`**, modify: **`PUT`**, delete: **`DELETE`**)
  - upon a successful operation the response code is **`CREATED`** (201) for creation, **`OK`** (200) or **`NO CONTENT`** (204) for other operations
    - with the **`CREATED`** result the created resource (object) and its location (**`GET`** path) must be returned as well
  - when the operation is not executed immediately, the status **`ACCEPTED`** (202) may be indicated
- in a RESTful service, the operations must conform to this scheme

DATA SYNCHRONIZATION MODELS

Client-side data management can be implemented in two ways:

- *synchronous:* client and server state are always the same
- *asynchronous:* client and server state are different and can be synchronized manually (save, load, update)

Asynchronous data management is advantageous if you want to save your changes in groups rather than individually

- to achieve this, changes have to be tracked on the client side with *status flags* and indicate what changes have been made to the data (new, modified, deleted)



ELTE EÖTVÖS LORÁND UNIVERSITY

TESTING WEB SERVICES

Modern web application development in .NET

TESTING WEB SERVICES

Testing of web services can be done

- manually, on the client side, using a program that sends the requests, such as a browser or target software (e.g. *Postman*, *Insomnia*, *Fiddler*)
- manually, using a tester desktop or web interface generated for the web service API (e.g. *Swagger UI*)
- automatically, on the client side, using a class that sends requests (e.g. using **`HttpClient`** in .NET)
- automatically, on the server side, by directly testing the controllers
  - the testing should be done in a managed environment
  - external factors (e.g. database) could be exluced, *mocked*

IN-MEMORY DATABASE

- The database context can be parameterized through dependency injection with the database engine to be used:

```
services.AddDbContext<MyDbContext>(options =>
  options.UseSqlServer(
    Configuration.GetConnectionString(
      "DefaultConnection")));
```

- Temporary in-memory database can also be used for testing:

```
var options = new DbContextOptionsBuilder<MyDbContext>()
  .UseInMemoryDatabase("TestDb").Options;
var context = new MyDbContext(options);
```

UNIT TEST FRAMEWORKS

For platform-independent unit tests, the most widely used frameworks are MSTest, NUnit or xUnit

- All three frameworks are natively supported by Visual Studio 2022.
- Most notable differences on a basic level:

MSTest	NUnit	xUnit	
`[TestClass]`	`[TestFixture]`	*n/a*	Test class
`[TestMethod]`	`[Test]`	`[Fact]`	Test case (method)
`[TestInitialize]`	`[Setup]`	*ctor*	Initialize test cases
`[TestCleanup]`	`[TearDown]`	`IDisposable`	Clean up test cases

MOCK OBJECTS

When testing a program unit with a dependency, we replace the dependency with a simulation of it, called a *mock object*

- the *mock object* implements the interface of the dependency, providing a simple, error-free functionality
- by using *mock objects*, the test actually checks the functionality of the specified program unit, unaffected by any error in the dependency

Mock objects can be created manually or by using a suitable software library

- e.g. *Moq*, *NSubstitute*, *FakeItEasy*, all avaliable as NuGet packages

MOCK OBJECTS

e.g.:

```
interface Idependency
{
  bool Check(double value);
  double Compute();
}
...
class DependencyImplementation : Idependency
{
  public bool Check(double value) { … }
  public double Compute() { … }
}
```

MOCK OBJECTS

e.g.:

```
class Dependant { // class with a dependency
  private IDependency _dependency;
  public Dependant(IDependency d) { _dependency = d; }
  // dependency injection
  ...
}

...

Dependant d = new Dependant(new DependencyImplementation());
// pass the implementation of the dependency
```

MOCK OBJECTS

e.g.:

```
class DependencyMock : IDependency // mock type {
  // simple implementation
  public double Compute() { return 1; }
  public bool Check(double value) {
    return value >= 1 && value <= 10;
  }
}

...

Dependant d = new Dependant(new DependencyMock());
// inject the mock object into the dependant
```

MOCK OBJECTS

Moq makes it easy to create mock objects from interfaces

- the **`Mock`** generic class can be used to instantiate the simulation, which can be accessed with the **`Object`** property and produces default behaviour, e.g:

```
Mock<IDependancy> mock = new Mock<IDependancy>();
// mock object of the dependency
Dependant d = new Dependant(mock.Object);
// can be used immediately
```

- the **`Setup`** method you can set the behaviour of any of its members (**`Returns(...)`**, **`Throws(...)`**, **`Callback(...)`**), parameters can be managed (It)

MOCK OBJECTS

- e.g.:

```
mock.Setup(obj => obj.Compute()).Returns(1);
// define the behaviour, always return 1
mock.Setup(obj =>
  obj.Check(It.IsInRange<Double>(0, 10, Range.Inclusive)))
    .Returns(true);

mock.Setup(obj => obj.Check(It.IsAny<Double>())
    .Returns(false);
// multiple cases based on the parameter
```

- method calls can also be traced whether occurred or not (**`Verify(...)`**), and how many times

MOCKING ENTITY FRAMEWORK

The interface of *Entity Framework* can be mocked as well in a similar way, for each **`DbSet`** property

- e.g.:

```
var cityData = new List<City> {
  new City { Id = 1, Name = "TESTCITY" }
};

IQueryable<City> qCityData = cityData.AsQueryable();
cityMock = new Mock<DbSet<City>>();
```

- In order to use the *mocked* **`DbSet`**, as an **`IEnumerable`** object, at least the **`ElementType`**, **`Expression`** and **`Provider`** properties, as well as the **`GetEnumerator()`** method must be configured.

MOCKING ENTITY FRAMEWORK

- e.g.:

```
cityMock.As<IQueryable<City>>().Setup(mock =>
  mock.ElementType).Returns(qCityData.ElementType);

cityMock.As<IQueryable<City>>().Setup(mock =>
  mock.Expression).Returns(qCityData.Expression);

cityMock.As<IQueryable<City>>().Setup(mock =>
  mock.Provider).Returns(qCityData.Provider);

cityMock.As<IQueryable<City>>().Setup(mock =>
  mock.GetEnumerator()).Returns(cityData.GetEnumerator());
 // return the previously defined list
```

MOCKING ENTITY FRAMEWORK

- setting up data modifier and deleting methods:

```
_buildingMock.Setup(mock =>
  mock.Add(It.IsAny<Building>()))
      .Callback<Building>( building => {
        buildingData.Add(building); }
); // set up the Add() method
_buildingMock.Setup(mock =>
  mock.Remove(It.IsAny<Building>()))
      .Callback<Building>(building => {
        buildingData.Remove(building);
 });
); // set up the Remvoe() method
```

INTEGRATION TESTING

Testing the combined behaviour of multiple software components is called an *integration test*. The elimination of external factors, such as errors due to network connectivity, should also be addressed when testing the client and server together.

- Since the service requires a web server, *ASP.NET Core* provides a lightweight web server (**`TestServer`**) that allows the service to run directly in memory without the need for a network connection
- The easiest way to create the test server is to use the **`WebApplicationFactory`** class, which is generically parameterized with the entry point of the web service to be tested, the **`Program`** class
    - We can also specify modifications of the configuration (e.g. in-memory database), in the **`ConfigureTestServices()`** method

INTEGRATION TESTING

- The client (a **`HttpClient`** object) can be instantiated with the **`CreateClient()`** method
  - client objects created this way are automatically connected to the test server instance,
  - all client requests are executed in memory.